# Contents



# sop-agent-rag — End-to-End Walkthrough

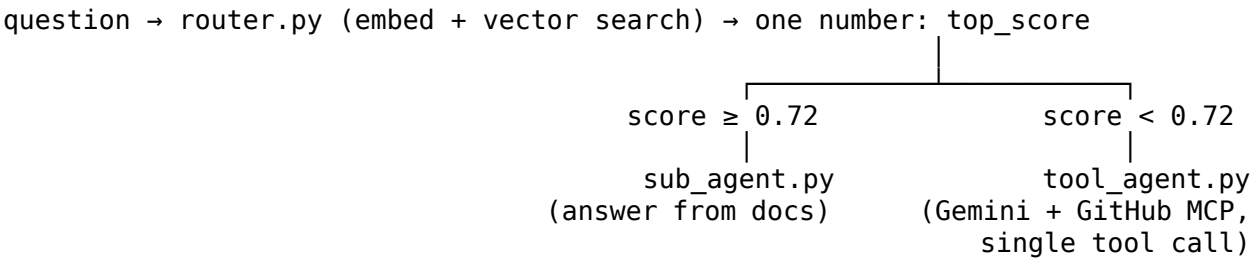
## 1. What this project is

A question-answering system for technical/SOP documents. You upload PDFs, the system chunks and embeds them, stores them in a vector database (Pinecone), and answers questions by finding relevant chunks and having an LLM (Gemini) write an answer grounded in them. That's classic RAG (Retrieval-Augmented Generation).

The mentor's ask was to evolve this from "one fixed pipeline" into "agentic orchestration" — a system that can pull from multiple data sources and *reason* about which ones to use and how to combine them, rather than following one hardcoded path every time.

---

## 2. What it looked like before



agents/graph.py (old version):

```
def decide_next(state):
    if state["top_score"]<0.72:
        return "tool_agent"
    return "sub_agent"

graph.add_conditional_edges("router", decide_next, {
    "sub_agent": "sub_agent",
    "tool_agent": "tool_agent"
})
```

The entire "decision" in this system was one `if` statement on one number. There was no reasoning — just a threshold. `tool_agent.py` could call *one* GitHub tool, once, then had to answer — no ability to look at the result and decide it needed more information.

---

## 3. What changed

| Before | After |
|---|---|
| `router.py` computes a score and **decides** the path | `router.py` only fetches document chunks (via `doc_tool.py`); it no longer decides anything |
| Two separate agents (`sub_agent`, `tool_agent`), picked by threshold | One agent (`orchestrator.py`) that can use **any** tool, in any combination |
| One tool call max, no follow-up | Up to 5 rounds of tool calls — it can look at a result and decide to call another tool |
| Only GitHub as an external source | GitHub **+** a new mock ERP system **+** the docs, all as equal-status tools |
| No visibility into *why* an answer was given | `decision_trace` — a log of every tool called, with what arguments and what came back |

In one sentence: the fixed `if/else` was replaced by a second agent whose entire job is to *decide* — same idea you identified earlier in this conversation.

---

## 4. New/changed files, in full, with concepts explained

### 4.1 `tools/erp_data.json` — the fake company database

```
{
  "employees": [
    {"employee_id": "E001", "name": "Priya Sharma", "department": "Engineering", "email":
    {"employee_id": "E002", "name": "Tom Reilly", "department": "Sales", "email": "tom.rei
    {"employee_id": "E003", "name": "Aisha Khan", "department": "IT Support", "email": "ai
    {"employee_id": "E004", "name": "Marco Silva", "department": "Finance", "email": "marc
  ],
  "assets": [
    {"asset_id": "A100", "type": "Laptop", "model": "Dell Latitude 5440", "assigned_to": "
```

```
    {"asset_id": "A101", "type": "Laptop", "model": "MacBook Pro 14", "assigned_to": "E002
    {"asset_id": "A102", "type": "Monitor", "model": "Dell U2723QE", "assigned_to": "E001"
    {"asset_id": "A103", "type": "Laptop", "model": "Dell Latitude 5440", "assigned_to": "
  ],
  "tickets": [
    {"ticket_id": "T500", "employee_id": "E001", "subject": "Account locked out after pass
    {"ticket_id": "T501", "employee_id": "E002", "subject": "VPN connection drops intermit
    {"ticket_id": "T502", "employee_id": "E001", "subject": "Request for second monitor",
    {"ticket_id": "T503", "employee_id": "E004", "subject": "Laptop replacement request",
  ]
}
```

**Concept — why fake data:** real ERP systems (SAP, Oracle, Odoo) need authenticated API access. We simulate the *shape* of that data — three related tables, linked by employee_id/assigned_to, exactly like foreign keys in a real database — so the rest of the system doesn't need to know or care that it's not talking to a real ERP.

### 4.2 `tools/erp_tool.py` — turning that data into callable functions

```python
# Mock ERP tool — a pretend company database (employees, equipment,
# support tickets) read from erp_data.json instead of a real system.
#
# Every function here returns a dict, never raises an error. If something
# isn't found we return {"found": False, ...} so the AI agent calling this
# can react to it instead of the program crashing.

import json
import os

DATA_FILE = os.path.join(os.path.dirname(__file__), "erp_data.json")

with open(DATA_FILE, "r") as f:
    DB = json.load(f)


def find_employee(name):
    search_term = name.strip().lower()

    for employee in DB["employees"]:
        employee_id = employee["employee_id"].lower()
        employee_name = employee["name"].lower()

        if search_term == employee_id:
            return employee
        if search_term in employee_name:
            return employee

    return None


def get_employee_info(name):
    employee = find_employee(name)
    if employee is None:
```

```
        return {"found": False, "message": f"No employee matching '{name}'"}
    return {"found": True, "employee": employee}


def get_tickets(name, status=None):
    employee = find_employee(name)
    if employee is None:
        return {"found": False, "message": f"No employee matching '{name}'"}

    matching_tickets = []
    for ticket in DB["tickets"]:
        if ticket["employee_id"] == employee["employee_id"]:
            matching_tickets.append(ticket)

    if status is not None:
        filtered_tickets = []
        for ticket in matching_tickets:
            if ticket["status"].lower() == status.lower():
                filtered_tickets.append(ticket)
        matching_tickets = filtered_tickets

    return {"found": True, "employee": employee["name"], "tickets": matching_tickets}


def get_assets(name):
    employee = find_employee(name)
    if employee is None:
        return {"found": False, "message": f"No employee matching '{name}'"}

    matching_assets = []
    for asset in DB["assets"]:
        if asset["assigned_to"] == employee["employee_id"]:
            matching_assets.append(asset)

    return {"found": True, "employee": employee["name"], "assets": matching_assets}
```

**Concepts:**

- `os.path.join(os.path.dirname(__file__), "erp_data.json")` — builds an absolute path to the data file so this code works regardless of which folder you launch the app from.
- `with open(...) as f:` — a context manager; the file is guaranteed to close automatically.
- `json.load(f)` — parses the file's text into real Python dicts/lists.
- **The "never crash" rule**: every function returns a plain dict, even for "not found." This matters specifically because the *caller* of these functions is going to be an LLM deciding what to do next — an unhandled exception would kill the whole reasoning process instead of letting the agent react to "not found."
- Plain `for` loops with `.append()` build filtered lists step by step — the explicit version of what more experienced Python devs often write as a one-line "list comprehension."

### 4.3 `tools/doc_tool.py` — wrapping your existing RAG search as a tool

```python
# Wraps your existing document search (agents/router.py's get_chunks) as
# a tool the orchestrator can choose to call — same status as the ERP
# functions. This is the change that removes the hardcoded score check:
# instead of router.py deciding automatically, the orchestrator now
# decides whether document search is even worth calling.

from agents.router import get_chunks


def search_docs(query, count=5):
    try:
        chunks = get_chunks(query, chunk_count=count)
    except Exception as e:
        return {"found": False, "message": f"Document search failed: {e}"}

    if not chunks:
        return {"found": False, "message": "No matching content found in the documents."}

    return {"found": True, "chunks": chunks}
```

**Concept:** this is the single most important structural change. Your original `get_chunks` (in `router.py`) ran *automatically* on every question. Now it only runs when the orchestrator's reasoning decides it's relevant — same underlying Pinecone search, but no longer privileged over the other tools.

### 4.4 `agents/router.py` — trimmed down

Still contains `get_chunks` (embeds the question, queries Pinecone, returns matching chunks with `document_name`, `chunk_text`, `similarity`). The `router_node`/`decide_next` threshold logic was deleted — nothing calls it anymore, and it referenced state fields (`chunks`, `top_score`) that no longer exist.

### 4.5 `agents/orchestrator.py` — the core of the whole project

This is the reasoning agent. Full file, then concept-by-concept:

```python
import os
import asyncio
from dotenv import load_dotenv
load_dotenv()

from google import genai
from google.genai import types

from tools.doc_tool import search_docs
from tools.erp_tool import get_employee_info, get_tickets, get_assets

GITHUB_MCP_URL = "https://api.githubcopilot.com/mcp/"
GITHUB_PAT = os.environ["GITHUB_PAT"]

MAX_STEPS = 5
```

```
LOCAL_TOOLS = {
    "search_docs": search_docs,
    "get_employee_info": get_employee_info,
    "get_tickets": get_tickets,
    "get_assets": get_assets,
}

LOCAL_DECLARATIONS = [
    types.FunctionDeclaration(
        name="search_docs",
        description="Search the uploaded technical/SOP documents for information relevant
        parameters={
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "What to search for in the docu
            },
            "required": ["query"],
        },
    ),
    types.FunctionDeclaration(
        name="get_employee_info",
        description="Look up an employee's basic info (department, email) by name.",
        parameters={
            "type": "object",
            "properties": {
                "name": {"type": "string", "description": "Employee name or ID"},
            },
            "required": ["name"],
        },
    ),
    types.FunctionDeclaration(
        name="get_tickets",
        description="Look up an employee's IT support tickets, optionally filtered by stat
        parameters={
            "type": "object",
            "properties": {
                "name": {"type": "string", "description": "Employee name or ID"},
                "status": {"type": "string", "description": "Optional: 'Open' or 'Closed'"
            },
            "required": ["name"],
        },
    ),
    types.FunctionDeclaration(
        name="get_assets",
        description="Look up equipment (laptops, monitors, etc.) assigned to an employee."
        parameters={
            "type": "object",
            "properties": {
                "name": {"type": "string", "description": "Employee name or ID"},
            },
            "required": ["name"],
        },
    ),
```

```
]

UNSUPPORTED_KEYS = {"additionalProperties", "dependentRequired", "$schema", "examples"}

def clean_schema(schema):
    if not isinstance(schema, dict):
        return schema
    cleaned = {}
    for key, value in schema.items():
        if key.startswith("x-") or key in UNSUPPORTED_KEYS:
            continue
        if isinstance(value, dict):
            cleaned[key] = clean_schema(value)
        elif isinstance(value, list):
            cleaned[key] = [clean_schema(v) if isinstance(v, dict) else v for v in value]
        else:
            cleaned[key] = value
    return cleaned

SYSTEM_PROMPT = (
    "You are a helpful assistant with access to multiple tools: searching internal "
    "technical/SOP documents, looking up employee/ticket/asset records in the "
    "company ERP, and GitHub tools for repositories, code, and issues. "
    "Decide which tool(s), if any, are relevant to the user's question. You may "
    "call more than one tool, one at a time, before giving your final answer. If "
    "no tool is relevant, answer directly from your own knowledge. Always base "
    "your final answer on the tool results you gathered, and say which source "
    "each piece of information came from."
)


async def run(question, chat_history=None):
    history_text = ""
    if chat_history:
        for turn in chat_history:
            history_text = history_text + f"{turn['role']}:{turn['content']}\n"

    contextual_question = question
    if history_text:
        contextual_question = f"Previous conversation:\n{history_text}\nCurrent question:

    from mcp import ClientSession
    from mcp.client.streamable_http import streamablehttp_client

    client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

    async with streamablehttp_client(
        GITHUB_MCP_URL,
        headers={"Authorization": f"Bearer {GITHUB_PAT}"},
    ) as (read, write, _):
        async with ClientSession(read, write) as session:
            await session.initialize()
```

```
            mcp_tools = (await session.list_tools()).tools
            mcp_declarations = []
            for tool in mcp_tools:
                try:
                    mcp_declarations.append(types.FunctionDeclaration(
                        name=tool.name,
                        description=tool.description or "",
                        parameters=clean_schema(tool.inputSchema),
                    ))
                except Exception as e:
                    print(f"Skipping tool '{tool.name}': {e}")

            mcp_tool_names = {tool.name for tool in mcp_tools}

            all_tools = types.Tool(function_declarations=LOCAL_DECLARATIONS + mcp_declarat

            chat = client.chats.create(
                model="gemini-2.5-flash",
                config=types.GenerateContentConfig(
                    tools=[all_tools],
                    system_instruction=SYSTEM_PROMPT,
                ),
            )

            decision_trace = []
            response = chat.send_message(contextual_question)

            for step in range(MAX_STEPS):
                part = response.candidates[0].content.parts[0]

                if not part.function_call:
                    return response.text, decision_trace

                tool_name = part.function_call.name
                tool_args = dict(part.function_call.args)

                if tool_name in LOCAL_TOOLS:
                    result = LOCAL_TOOLS[tool_name](**tool_args)
                elif tool_name in mcp_tool_names:
                    tool_result = await session.call_tool(tool_name, tool_args)
                    result = {"result": tool_result.content[0].text}
                else:
                    result = {"error": f"Unknown tool '{tool_name}'"}

                decision_trace.append({
                    "step": step + 1,
                    "tool": tool_name,
                    "args": tool_args,
                    "result": result,
                })

                response = chat.send_message(
                    types.Part.from_function_response(name=tool_name, response=result)
```

```
                )

            return "Reached the reasoning step limit without a final answer.", decision_tr


def orchestrator_node(state):
    answer, decision_trace = asyncio.run(run(state["question"], state.get("chat_history"))
    return {"answer": answer, "decision_trace": decision_trace}
```

**Concept: LLM function/tool calling** Gemini can't inspect your Python code. `LOCAL_DECLARATIONS` is how you describe each tool to it — name, plain-English description, and a JSON schema for the arguments. Given a question, Gemini decides: "this needs `get_tickets` with `name='Priya Sharma'`, `status='Open'`" and returns that as a structured `function_call` instead of text. Your code then actually executes the real Python function — the model never runs code itself, it just *requests* that you run something and hands you the arguments.

`LOCAL_TOOLS` (name → real function) and `LOCAL_DECLARATIONS` (name → description Gemini sees) are two views of the same four tools, kept in sync by name.

**Concept: MCP (Model Context Protocol)** GitHub exposes its tools (search repos, read issues, etc.) over MCP — a standard way for a tool provider to describe and serve tools over a network connection, instead of you writing bespoke integration code for every GitHub API endpoint. `streamablehttp_client` opens that connection; `ClientSession` is the conversation with GitHub's MCP server; `session.list_tools()` asks it "what can you do?"; `session.call_tool(name, args)` actually invokes one. `clean_schema` exists because GitHub's tool descriptions include some JSON-schema fields Gemini's function calling doesn't support, so we strip them before handing schemas to Gemini.

```
for step in range(MAX_STEPS):
    part = response.candidates[0].content.parts[0]
    if not part.function_call:
        return response.text, decision_trace
    ...run the tool, get `result`...
    response = chat.send_message(types.Part.from_function_response(name=tool_name, respons
```

**Concept: the ReAct reasoning loop (the actual "agentic" part) ReAct = Reason, Act, Observe, repeat.** Each pass through this loop is one cycle: Gemini *reasons* about what it needs, *acts* by requesting a tool call, your code runs it and hands back the *observation* (the result) via `chat.send_message(...)`. Gemini then reasons again — now armed with that new information — and either asks for another tool or returns a final text answer. `client.chats.create(...)` is what makes this possible: unlike a single `generate_content` call, a `chat` object remembers the whole conversation automatically, so each `send_message` builds on everything before it.

`MAX_STEPS = 5` is a safety valve — without it, a confused model could theoretically loop forever, burning API calls.

```
if tool_name in LOCAL_TOOLS:
    result = LOCAL_TOOLS[tool_name](**tool_args)
elif tool_name in mcp_tool_names:
```

```
    tool_result = await session.call_tool(tool_name, tool_args)
    result = {"result": tool_result.content[0].text}
```

**Concept: uniform dispatch over two different tool types** Local tools (ERP, docs) run instantly, in this same process. GitHub tools require an actual network round-trip through the MCP session. Gemini doesn't know or care about that difference — from its side, it just asked for a tool and got a result. This `if/elif` is where your code translates "Gemini wants X" into "here's how to actually get X."

**Concept: `decision_trace` — the transparency layer** Every tool call gets logged: which tool, what arguments, what came back. This is what answers "how did the agent decide?" — you can show this to your mentor as literal proof of the reasoning path for any given question, not just the final answer.

### 4.6 `agents/state.py` — what data flows through the graph

```
from typing import TypedDict, List

class GraphState(TypedDict):
    question: str
    answer: str
    chat_history: List[dict]
    decision_trace: List[dict]
```

`chunks` and `top_score` (old, router-decides-everything fields) are gone; `decision_trace` is new. A `TypedDict` is just a dict with declared key names/types for clarity — Python won't enforce it at runtime, but it documents the shape of state flowing through the graph and helps tooling/autocomplete.

### 4.7 `agents/graph.py` — the graph itself, drastically simplified

```
from langgraph.graph import StateGraph, START, END
from .state import GraphState
from .orchestrator import orchestrator_node

graph = StateGraph(GraphState)
graph.add_node("orchestrator", orchestrator_node)

graph.add_edge(START, "orchestrator")
graph.add_edge("orchestrator", END)

compiled_graph = graph.compile()
```

Before: 3 nodes, a conditional edge, a threshold. Now: 1 node, 2 fixed edges. **This is the concrete proof of "the if/else got replaced by a deciding agent"** — LangGraph itself makes zero decisions now; it's pure entry/exit scaffolding. All the actual judgment happens inside `orchestrator_node`.

### 4.8 `main.py` — one line changed

```python
@app.post("/api/chat")
def chat(data:dict):
    result=compiled_graph.invoke({"question":data["message"],"chat_history":data.get("chat
    return {"response":result["answer"],"decision_trace":result.get("decision_trace",[])}
```

The API now returns `decision_trace` alongside the answer, so the frontend (or you, inspecting the response) can see which tools fired for any given question.

#### 4.9 Deleted: `agents/sub_agent.py`, `agents/tool_agent.py`

Fully superseded by `orchestrator.py` — nothing imports them anymore.

---

## 5. Deep dive: structured tool calling (the agent deciding on its own)

"Structured tool calling" (also called function calling) is the mechanism that lets Gemini *decide* which tool to use, instead of you writing `if "ticket" in question: call get_tickets()` yourself. Here's exactly how that decision happens, mechanically.

#### 5.1 The schema is the contract

```python
types.FunctionDeclaration(
    name="get_tickets",
    description="Look up an employee's IT support tickets, optionally filtered by status (
    parameters={
        "type": "object",
        "properties": {
            "name": {"type": "string", "description": "Employee name or ID"},
            "status": {"type": "string", "description": "Optional: 'Open' or 'Closed'"},
        },
        "required": ["name"],
    },
),
```

This is a JSON Schema — the same format used for API validation everywhere, not something specific to Gemini. It says: this tool is called `get_tickets`, it does *this* (the `description` — Gemini reads this like a sentence to decide relevance), and it accepts two arguments, `name` (required, a string) and `status` (optional, a string). Every one of the four `LOCAL_DECLARATIONS` plus every GitHub MCP tool's schema get bundled into one list and handed to Gemini via:

```python
all_tools = types.Tool(function_declarations=LOCAL_DECLARATIONS + mcp_declarations)

chat = client.chats.create(
    model="gemini-2.5-flash",
    config=types.GenerateContentConfig(
        tools=[all_tools],
        system_instruction=SYSTEM_PROMPT,
    ),
)
```

At this point Gemini has, in effect, a menu: N tools, each with a name, a description, and a strict shape for arguments.

### 5.2 Why "structured" — the alternative, and why it's worse

The naive approach would be: ask Gemini in plain text "which tool should I use, and with what arguments?", get back a sentence like *"You should call get_tickets for Priya Sharma with status Open"*, and then write fragile regex/string-parsing code to extract `get_tickets`, `Priya Sharma`, and `Open` back out of that sentence. That breaks the moment Gemini phrases it slightly differently.

Structured/function calling skips all of that: Gemini is constrained (by the API itself, not just prompting) to return either normal text, or a `function_call` object with a `.name` (must match one of the declared tool names) and `.args` (must match the declared schema's types/required fields). Your code never parses natural language to figure out what to do — it reads a typed object:

```
part = response.candidates[0].content.parts[0]

if not part.function_call:
    return response.text, decision_trace   # Gemini answered in plain text — no tool neede

tool_name = part.function_call.name        # e.g. "get_tickets"
tool_args = dict(part.function_call.args)  # e.g. {"name": "Priya Sharma", "status": "Open
```

**This is the actual moment the agent "decides."** Given the question and the tool menu, Gemini's own reasoning (you never see it directly, it happens inside the model) picks zero, one, or — across loop iterations — several tools, and `part.function_call` is simply whichever one it picked, if any. `not part.function_call` being True means "I decided no tool is needed, here's my answer" — that's a decision too, just one that doesn't require external data.

### 5.3 From decision to execution

Because `tool_args` is already a plain dict with the right keys, calling the real function is a one-liner:

```
if tool_name in LOCAL_TOOLS:
    result = LOCAL_TOOLS[tool_name](**tool_args)
```

`**tool_args` unpacks the dict into keyword arguments — `{"name": "Priya Sharma", "status": "Open"}` becomes the actual call `get_tickets(name="Priya Sharma", status="Open")`. This only works cleanly *because* the schema in `LOCAL_DECLARATIONS` was written to use the exact same parameter names as the real Python functions in `erp_tool.py`/`doc_tool.py` — that alignment is deliberate, not automatic.

---

## 6. Deep dive: `decision_trace` — logging every decision

### 6.1 What gets recorded, and when

Inside the loop, right after a tool runs and before its result is sent back to Gemini:

```
decision_trace.append({
    "step": step + 1,
    "tool": tool_name,
    "args": tool_args,
    "result": result,
})
```

Every entry is a plain dict with four fields — `step` (which iteration of the loop this was, 1-indexed for readability), `tool` (the name Gemini chose), `args` (exactly what it decided to pass in), and `result` (exactly what came back). Because `decision_trace` is a list and each entry is a small flat dict, the whole thing is directly JSON-serializable — it can be returned from an API, logged to a file, or rendered in a UI with zero conversion work.

### 6.2 Walking through a real single-tool example

For the question *"Does Priya Sharma have any open tickets?"*, here's what actually happens, step by step (this is real output from testing the loop logic):

1. `chat.send_message("Does Priya Sharma have any open tickets?")` — Gemini reasons about the question against the tool menu, decides `get_tickets` is relevant, returns a `function_call` (not text).
2. Loop iteration 1: `part.function_call` is truthy → `tool_name = "get_tickets"`, `tool_args = {"name": "Priya Sharma", "status": "Open"}`.
3. `LOCAL_TOOLS["get_tickets"](name="Priya Sharma", status="Open")` runs for real against `erp_data.json`, returns:

```
{"found": True, "employee": "Priya Sharma",
 "tickets": [{"ticket_id": "T500", "employee_id": "E001",
              "subject": "Account locked out after password reset",
              "status": "Open", "created_date": "2026-07-28"}]}
```

4. That whole dict gets appended to `decision_trace` as `{"step": 1, "tool": "get_tickets", "args": {...}, "result": {...}}`.
5. `chat.send_message(types.Part.from_function_response(name="get_tickets", response=result))` sends the result back into the conversation.
6. Loop iteration 2: Gemini now has the ticket data. It decides no further tool is needed — `part.function_call` is `None` this time — so `return response.text, decision_trace` fires, with `response.text` = *"Priya Sharma has 1 open ticket: account lockout after password reset."*

Final `decision_trace` for this question has exactly **one** entry — one tool, one decision.

### 6.3 What a multi-tool trace looks like

For a question like *"What's the SOP for a locked account, and does Priya Sharma have an open ticket for that?"*, the same loop just runs twice before Gemini is satisfied:

```
[
    {"step": 1, "tool": "search_docs", "args": {"query": "locked account SOP"},
     "result": {"found": True, "chunks": [...]}},
    {"step": 2, "tool": "get_tickets", "args": {"name": "Priya Sharma", "status": "Open"},
     "result": {"found": True, "employee": "Priya Sharma", "tickets": [...]}},
]
```

Nothing in the loop's code changes between the one-tool and two-tool case — `for step in range(MAX_STEPS)` just keeps iterating as long as Gemini keeps returning `function_calls` instead of final text. This is the concrete difference from the old `tool_agent.py`, which physically could not do this — it called at most one tool, once, with no way to loop back.

### 6.4 Where the trace goes after the loop

```
def orchestrator_node(state):
    answer, decision_trace = asyncio.run(run(state["question"], state.get("chat_history"))
    return {"answer": answer, "decision_trace": decision_trace}
```

`orchestrator_node` is the function LangGraph actually calls (see `agents/graph.py`) — it hands back a dict that gets merged into `GraphState`, which declares `decision_trace: List[dict]` as one of its fields. From there, `main.py`'s `/api/chat` endpoint reads it straight off the result:

```
result = compiled_graph.invoke({"question": ..., "chat_history": ...})
return {"response": result["answer"], "decision_trace": result.get("decision_trace", [])}
```

So the full path is: **Gemini's decision → `part.function_call` → tool execution → `decision_trace.append(...)` → `orchestrator_node`'s return dict → `GraphState` → `main.py`'s JSON response.** Nothing in between that path throws the trace away — a decision made at step 1 of the loop is still visible in the final HTTP response.

---

## 7. What kinds of questions it can now answer

Because the orchestrator picks tools dynamically, the same endpoint now handles several categories of question, dispatched automatically:

**Document/SOP questions** (uses `search_docs`)

- "What's the SOP for resetting a locked account?"
- "Explain the steps for onboarding a new device."

**ERP questions** (uses `get_employee_info` / `get_tickets` / `get_assets`)

- "What department is Priya Sharma in?"
- "Does Tom Reilly have any open support tickets?"
- "What laptop is assigned to Marco Silva?"

**GitHub questions** (uses GitHub's MCP tools, unchanged from before)

- "Search GitHub for the langgraph repository and summarize it."
- "What are the open issues on <repo>?"

**General knowledge** (no tool at all)

- "What does REST stand for?" — Gemini just answers directly if no tool is relevant.

**Multi-tool questions** — the actual new capability, impossible before

- "What's the SOP for a locked account, and does Priya Sharma currently have an open ticket for that?" → calls `search_docs` **and** `get_tickets`, then combines both in one answer.
- "Priya Sharma reported a laptop issue — what equipment does she have, and is there a relevant troubleshooting doc?" → `get_assets` + `search_docs`.

This last category is the actual deliverable for your mentor: a single agent reasoning across multiple, independent data sources in one response, with a visible trace of how it got there.

---

## 8. Current status / what's left

- All code committed locally (commit `4a4fa15` on `main`) — not yet pushed (auth needs your own GitHub credentials; the `.env` token wasn't scoped for git push).
- Logic verified via mocked tests in a sandboxed environment (real network to Hugging Face/Gemini/GitHub was blocked there) — needs a real run on your machine (`uvicorn main:app --reload`) to confirm live behavior end-to-end.
- Next: `git push` from your terminal → confirm Render redeploys → test the multi-tool questions above against the live app.